Machine Learning — Assignment 2

M.Tech (AIML) · Work Integrated Learning Programmes, BITS Pilani

Credit Card Default Prediction with Five Classifiers and a Streamlit App

Name: Himanshu Sharma | BITS ID: _____

1. GitHub Repository Link

https://github.com/hk9454/assignment

Contains the complete source code, requirements.txt, README.md, test_data.csv and the model/ folder with all five saved classifiers.

2. Live Streamlit App Link

<Streamlit Community Cloud link to be added>

3. Screenshot — Execution on BITS Virtual Lab

BITS Virtual Lab screenshot to be inserted here

4. GitHub README Content

Credit Card Default Prediction — ML Assignment 2

M.Tech (AIML) · Machine Learning · Assignment 2

- **GitHub repository:** https://github.com/hk9454/assignment
- **Live Streamlit app:** `[Streamlit app link to be added]`

a. Problem statement

A credit-card issuer wants to know, **before the next billing cycle**, which clients are likely to miss their next payment. Given a client's credit limit, demographics and the last six months of repayment status, bill amounts and payment amounts, predict whether the client will **default on the payment next month** (binary classification: `1` = default, `0` = no default).

Five classifiers (Logistic Regression, Decision Tree, K-Nearest Neighbours, Gaussian Naive Bayes and Support Vector Machine) are trained on the same data and compared using Accuracy, AUC, Precision, Recall, F1 and Matthews Correlation Coefficient (MCC). The trained models are served through an interactive Streamlit app that evaluates them on uploaded test data without retraining.

b. Dataset description

Item	Value
Name	*Default of Credit Card Clients* (Yeh & Lien, 2009)
Source	UCI Machine Learning Repository, dataset id 350 — https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients
Instances	30,000 clients (Taiwan, April–September 2005)
Input features	23 (the `ID` column is dropped)
Target	`default` (originally `default payment next month`) — 0: 23,364 (77.9 %), 1: 6,636 (22.1 %)
Missing values	None

Features

Feature(s)	Meaning
`LIMIT_BAL`	Amount of credit granted (NT$), individual + family/supplementary
`SEX`	1 = male, 2 = female
`EDUCATION`	1 = graduate school, 2 = university, 3 = high school, 4 = others
`MARRIAGE`	1 = married, 2 = single, 3 = others
`AGE`	Age in years
`PAY_0`, `PAY_2` … `PAY_6`	Repayment status Sep → Apr (-2/-1 = paid duly / no use, 1…9 = months of delay)
`BILL_AMT1` … `BILL_AMT6`	Bill statement amount Sep → Apr (NT$)
`PAY_AMT1` … `PAY_AMT6`	Amount paid Sep → Apr (NT$)

Preprocessing (inside each saved pipeline, see `src/features.py` and `src/train.py`)

1. Undocumented category codes are folded into "others": `EDUCATION` $\in$ `{0,5,6}` → `4`, `MARRIAGE = 0` → `3`.
2. Three engineered features: `UTIL_1` (latest bill / credit limit), `REPAY_RATIO_1` (latest payment / previous bill), `MAX_DELAY` (worst repayment status over six months).
3. Numeric columns are standardised; `SEX`, `EDUCATION`, `MARRIAGE` are one-hot encoded.

4. Stratified 80 / 20 train–test split (`random_state = 26`). The 6,000-row test split is saved as `test_data.csv`.

c. GitHub repository

https://github.com/hk9454/assignment

```
|-- app.py                  # Streamlit app
|-- requirements.txt
|-- README.md
|-- test_data.csv           # 20 % hold-out test split (6,000 rows, features + target)
|-- data/credit_default.csv # full dataset (CSV export of the UCI .xls)
|-- model/                  # saved pipelines (.joblib) for all five classifiers + metrics.csv
`-- src/
    |-- features.py         # feature list, cleaning, feature engineering
    |-- metrics.py          # the six evaluation metrics
    `-- train.py            # trains and saves all models
```

Reproduce:

```
pip install -r requirements.txt
python -m src.train        # retrains models, rewrites model/ and test_data.csv
streamlit run app.py
```

d. Models used

All five models are trained on the same training split and evaluated on the same 6,000-row test split (`test_data.csv`) with a 0.5 decision threshold. Positive class = default (1).

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.8060	0.7407	0.6646	0.2479	0.3611	0.3204
Decision Tree	0.8150	0.7590	0.6662	0.3278	0.4394	0.3747
KNN	0.8097	0.7436	0.6290	0.3399	0.4413	0.3620
Naive Bayes	0.6630	0.7328	0.3641	0.7016	0.4794	0.2968
Support Vector Machine	0.8142	0.7148	0.6743	0.3090	0.4238	0.3667

Hyper-parameters: Logistic Regression (`C=0.5`, L2), Decision Tree (`max_depth=6`, `min_samples_leaf=40`), KNN (`k=25`, distance-weighted), Gaussian Naive Bayes (defaults), SVM (RBF kernel, `C=1`, Platt-scaled probabilities via `CalibratedClassifierCV`).

Observations

ML Model Name	Observation about model performance
Logistic Regression	Solid, well-calibrated baseline (AUC 0.74) but the lowest recall (0.25): a linear boundary on the class-imbalanced data pushes most borderline clients to "no default". Good when false alarms are expensive, weak at catching defaulters.
Decision Tree	Best overall: highest Accuracy (0.815), AUC (0.759) and MCC (0.375). A shallow tree captures the strong non-linear rule "recent payment delay ⇒ high risk" (`PAY_0`, `MAX_DELAY`) and stays interpretable; depth and leaf-size limits prevent overfitting.
KNN	Close to the tree on Accuracy and F1 (0.441) with slightly better recall (0.34), but lower precision and MCC. It is the heaviest model to store (keeps all training points) and slowest at prediction time.
Naive Bayes	Very different trade-off: highest recall (0.70) and F1 (0.48) but by far the lowest Accuracy (0.66), precision (0.36) and MCC (0.30). The independence assumption breaks on the strongly correlated `BILL_AMT*` / `PAY_*` series, so it over-predicts default. Useful only as a high-sensitivity screen.
Support Vector Machine	Accuracy (0.814) and precision (0.674, the best) match the tree, but recall is low (0.31) and AUC is the lowest (0.715), meaning its probability ranking is weaker; it is also the slowest model to train.

ML Model Name	Observation about model performance
Overall winner	**Decision Tree** — best Accuracy, AUC and MCC (MCC is the most reliable single metric for this imbalanced target). All models are limited by the 78/22 class imbalance; lowering the decision threshold (available as a slider in the app) trades precision for recall if catching defaulters matters more.

Streamlit app

Features:

- **CSV upload** of test data (or toggle to use the bundled `test_data.csv`, which can also be downloaded from the sidebar)
- **Model selection** dropdown for the five saved models
- **Evaluation metrics**: Accuracy, AUC, Precision, Recall, F1, MCC
- **Confusion matrix**, **classification report** and ROC curve
- **Compare all models** tab with a metrics table and bar chart
- Adjustable decision threshold and downloadable predictions

The app only loads the saved models from `model/` and predicts; it never retrains on uploaded data. The uploaded CSV must contain the 23 feature columns listed above and the `default` target column.